How to use a Web API in Dart

Example:

In a dart application find the current exchange rate from EUR to USD or CHF

Gemini’s answer under https://gemini.google.com/share/2d40b1fbf140

Write a Dart app that finds out the current exchange rate from EUR to USD

Building a Dart application to fetch exchange rates is a classic way to practice working with APIs and JSON parsing.

To keep this simple and functional without requiring you to sign up for an API key, we'll use the **Frankfurter API**. It’s open-source, free, and doesn’t require authentication for basic use.

“Frankfurter API” https://frankfurter.dev/

Frankfurter Exchange Rates API

fc60b61

Star 1,238

Frankfurter is a free, open-source currency data API that tracks reference exchange rates published by institutional and non-commercial sources like the European Central Bank.

No usage caps or API keys. Works great client-side in the browser or mobile apps. The public API is available at api.frankfurter.dev. If preferred, you can self-host.

API Endpoints

Frankfurter provides endpoints to retrieve latest rates, historical data, or time series.

ChatGPT

What does the abbreviation "API" stand for ?

API stands for **Application Programming Interface.**

In simple terms, it's a **set of rules that lets different software talk to each other.** Instead of building everything from scratch, one program can request data or actions from another via its API.

Example:

- A weather app uses a weather service's API to get current temperature.
- A website uses a payment API (like Stripe or PayPal) to process payments.
- Your phone uses APIs to connect apps with the operating system (camera, GPS, etc.).

Think of it like a **menu in a restaurant:**

- The menu = API (what you can request)
- The kitchen = the system doing the work
- You don't see how food is made — you just order and get results.

Documentation of “Frankfurter API” https://frankfurter.dev/

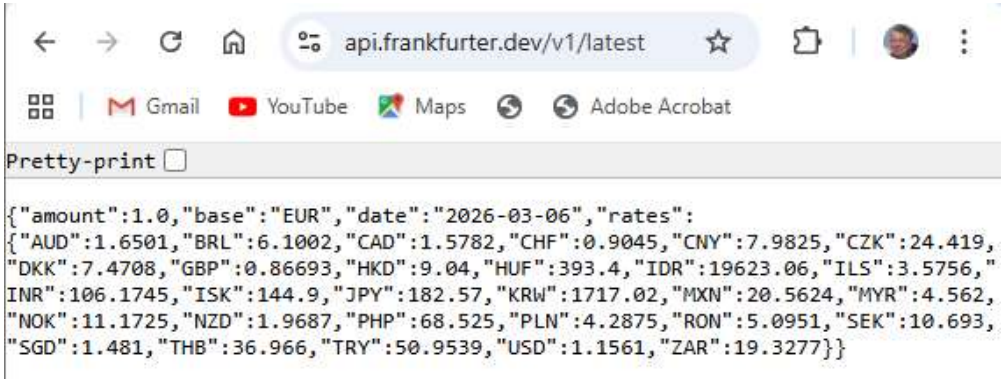
Latest Rates

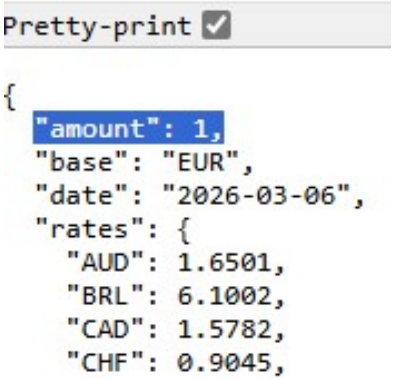
Fetch the latest working day's rates, updated daily around 16:00 CET.

```
/* curl -s https://api.frankfurter.dev/v1/latest */
{
  "base": "EUR",
  "date": "2026-03-06",
  "rates": {
    "AUD": 1.6501,
    "BRL": 6.1002,
    "CAD": 1.5782,
    "CHF": 0.9045,
    "...": "..."
  }
}
```

https://api.frankfurter.dev/v1/latest

Test in Chrome:





Documentation of “Frankfurter API” https://frankfurter.dev/

Change the base currency using the `base` parameter. The default is EUR.

```
/* curl -s https://api.frankfurter.dev/v1/latest?base=USD */
{
  "base": "USD",
  "date": "2026-03-06",
  "rates": {
    "AUD": 1.4273,
    "BRL": 5.2765,
    "CAD": 1.3651,
    "CHF": 0.78237,
    "...": "..."
  }
}
```

https://api.frankfurter.dev/v1/latest?base=USD

Limit the response to specific target currencies.

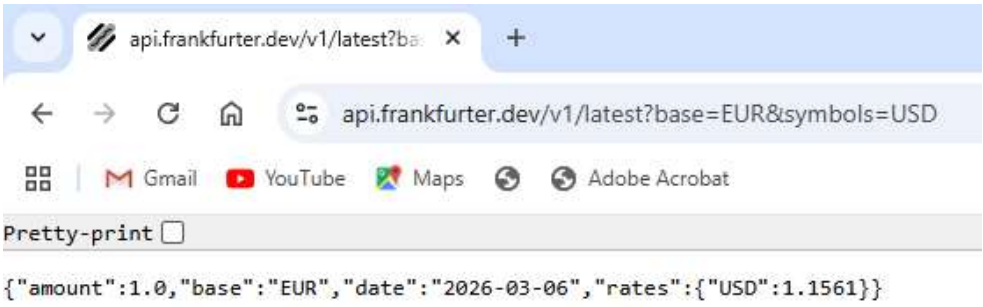
```
/* curl -s https://api.frankfurter.dev/v1/latest?symbols=CHF,GBP */
{
  "base": "EUR",
  "date": "2026-03-06",
  "rates": {
    "CHF": 0.9045,
    "GBP": 0.86693
  }
}
```

https://api.frankfurter.dev/v1/latest?symbols=CHF,GBP

Documentation of “Frankfurter API” https://frankfurter.dev/

In Gemini’s code proposal for our question on slide 1 the following URL is used:

https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD



“Frankfurter API”

“Frankfurter API” is a REST API:

REST stands for **Representational State Transfer**.

It isn’t a specific piece of software or a programming language; it is an **architectural style** for designing networked applications. It relies on a stateless, client-server communication protocol —almost always **HTTP**.

Think of REST as a set of "rules of the road" that allows different systems (like your Dart app and the exchange rate server) to talk to each other reliably.

Components of a REST API URL

Explain the components used for a REST API URL e.g. in https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD,CHF

A REST API URL (also known as an **Endpoint**) is structured to be machine-readable and hierarchical. Using your example, here is a breakdown of the components:

`https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD,CHF`

---

1. Scheme (Protocol)

`https://`

This defines how data is being transmitted.

- **HTTPS** (Hypertext Transfer Protocol Secure) is the industry standard for APIs because it encrypts the data sent between the client and the server.

2. Subdomain

`api.`

Many organizations host their API on a specific subdomain (like `api` or `dev`) to separate API traffic from their main website traffic (e.g., `www.frankfurter.dev`).

3. Host / Domain Name

`frankfurter.dev`

This is the address of the server where the API is hosted. It points to the specific entity providing the service.

4. Versioning

`/v1/`

API developers use versioning to ensure that when they make updates, they don't break existing applications using the older version. If the developer releases a massive update that changes the data structure, they would release a `/v2/`, allowing users to migrate at their own pace.

5. Path (Resource)

`/latest`

The path indicates the specific **resource** you are requesting. In REST logic, the path usually represents a noun. In this case, you are asking for the "latest" exchange rates. If you wanted historical data, the path might change to `/2023-01-01`.

Components of a REST API URL

https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD,CHF

5. Path (Resource)

`/latest`

The path indicates the specific **resource** you are requesting. In REST logic, the path usually represents a noun. In this case, you are asking for the "latest" exchange rates. If you wanted historical data, the path might change to `/2023-01-01`.

6. Query String Separator

`?`

The question mark acts as a divider. Everything before it is the location of the resource; everything after it is a set of instructions/filters for that resource.

7. Query Parameters

`base=EUR&symbols=USD,CHF`

These are "key-value pairs" used to filter or modify the data being returned.

- **Key:** `base` / **Value:** `EUR` (Tells the API to calculate rates starting from the Euro).
- **Key:** `symbols` / **Value:** `USD,CHF` (Tells the API to only return data for the Dollar and Swiss Franc).

8. Parameter Delimiter

`&`

The ampersand is used to separate multiple query parameters. It allows you to chain together many filters in a single request.

Summary Table

Component	Example from your URL	Purpose
Protocol	`https://`	Security and communication method.
Host	`api.frankfurter.dev`	Where the API lives.
Version	`/v1`	Ensures backward compatibility.
Endpoint/Path	`/latest`	The specific resource/action requested.
Parameters	`base=EUR`	Filters or specific options for the data.

Gemini's answer https://gemini.google.com/share/2d40b1fbf140 continued:

1. Project Setup

First, you'll need the `http` package. Add it to your `pubspec.yaml`:

YAML

```yaml
dependencies:
  http: ^1.1.0
```

Then run `dart pub get`.

2. The Dart Code

This script performs a GET request to the Frankfurter API, parses the JSON response, and extracts the EUR to USD rate.

Dart

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;
```

```dart
void main() async {
  await fetchExchangeRate();
}
```

```dart
Future<void> fetchExchangeRate() async {
  // We use Frankfurter API: no key needed for basic EUR -> USD
  final url = Uri.parse('https://api.frankfurter.dev/v1/latest?base=EUR&symbols=U

  try {
    print('Fetching current exchange rate...');
    final response = await http.get(url);

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);

      // The API returns a Map: { "base": "EUR", "date": "...", "rates": { "USD":
      double usdRate = data['rates']['USD'];
      String date = data['date'];

      print('--------------------------------');
      print('Base Currency: EUR');
      print('Target Currency: USD');
      print('Current Rate: \$${usdRate.toStringAsFixed(4)}');
      print('Last Updated: $date');
      print('--------------------------------');
    } else {
      print('Failed to load data. Status code: ${response.statusCode}');
    }
  } catch (e) {
    print('An error occurred: $e');
  }
}
```

http package on pub.dev https://pub.dev/packages/http/install

pub.dev Sign in Help

http 1.6.0

Published 3 months ago • dart.dev

SDK DART FLUTTER PLATFORM ANDROID IOS LINUX MACOS WEB WINDOWS 8.4K

Readme Changelog Example Installing Versions Scores

Use this package as a library

Depend on it

Run this command:

With Dart:

```
$ dart pub add http
```

With Flutter:

```
$ flutter pub add http
```

8.42k LIKES 160 POINTS 8.53M DOWNLOADS

Publisher

dart.dev

Weekly Downloads

2025.04.11 - 2026.03.06

Metadata

A composable, multi-platform, Future-based

File “pubspec.yaml” is needed

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\flutter\code\dart_advanced> dart pub add http
Found no `pubspec.yaml` file in `C:\flutter\code\dart_advanced` or parent directories
PS C:\flutter\code\dart_advanced>
```

There are 2 needed entries in pubspec.yaml:

```
pubspec.yaml
1   name: dart_advanced
2   environment:
3     sdk: ^3.9.2
4
```

```
PS C:\flutter\code\external_packages> dart --version
Dart SDK version: 3.9.2 (stable) (Wed Aug 27 03:49:40
```

```
PS C:\flutter\code\dart_advanced> dart pub add http
Resolving dependencies...
Downloading packages...
+ async 2.13.0
+ collection 1.19.1
+ http 1.6.0
+ http_parser 4.1.2
+ meta 1.18.1
+ path 1.9.1
+ source_span 1.10.2
+ string_scanner 1.4.1
+ term_glyph 1.2.2
+ typed_data 1.4.0
+ web 1.1.1
Changed 11 dependencies!
```

```
pubspec.yaml
1   name: dart_advanced
2   environment:
3     sdk: ^3.9.2
4   dependencies:
5     http: ^1.6.0
6
```

JSON Decoding in detail

```dart
final url = Uri.parse(
  'https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD',
);
```

https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD


```json
{
  "amount": 1,
  "base": "EUR",
  "date": "2026-03-06",
  "rates": {
    "USD": 1.1561
  }
}
```


```dart
try {
  print('Fetching current exchange rate...');
  final response = await http.get(url);

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);

    // The API returns a Map: { "base": "EUR", "date": "...", "rates": { "USD": 1.08 } }
    double usdRate = data['rates']['USD'];
    String date = data['date'];

    print('------------------------------');
    print('Base Currency: EUR');
    print('Target Currency: USD');
    print('Current Rate: \$${usdRate.toStringAsFixed(4)}');
    print('Last Updated: $date');
    print('------------------------------');
  } else {
    print('Failed to load data. Status code: ${response.statusCode}');
  }
} catch (e) {
  print('An error occurred: $e');
}
```

Enforcing some errors

```
try {
  print('Fetching current exchange rate...');
  final response = await http.get(url);

  if (response.statusCode == 200) {···
  } else {
    print('Failed to load data. Status code: ${response.statusCode}');
  }
} catch (e) {
  print('An error occurred: $e');
}
```

```
final url = Uri.parse(
  'https://api.frankfurter.dev/v1/latest?base=EUR&symbols=XXX',
);
```

```
Failed to load data. Status code: 404
```

```
final url = Uri.parse(
  'https://api.frankfurter.dev/v1/latestXX?base=EUR&symbols=USD',
);
```

```
Failed to load data. Status code: 404
```

```
final url = Uri.parse(
  'https://api.frankfurter.devXX/v1/latest?base=EUR&symbols=USD',
);
```

```
An error occurred: ClientException with SocketException: Failed host
lookup: 'api.frankfurter.devxx' (OS Error: No such host is known.
, errno = 11001), uri=https://api.frankfurter.devxx/v1/latest?base=E
UR&symbols=USD
```

No internet connection:

```
An error occurred: ClientException with SocketException: Failed host
lookup: 'api.frankfurter.dev' (OS Error: No such host is known.
, errno = 11001), uri=https://api.frankfurter.dev/v1/latest?base=EUR
&symbols=USD
```

http response status codes https://gemini.google.com/share/0901d8002228

Think of an **HTTP response status code** as a short-hand note from a server to your browser. Whenever you click a link or type a URL, the server sends back a three-digit code to tell you exactly how the request went—whether it was a "thumbs up," a "hang on a second," or a "total disaster."

2xx: Success

The gold standard. This means the action was received, understood, and accepted.

- **200 OK:** The standard "everything is fine" response.

4xx: Client Error

This is on you (the user). The request contains bad syntax or cannot be fulfilled.

- **400 Bad Request:** The server didn't understand what you wanted because the request was garbled.
- **401 Unauthorized:** You need to log in to see this.
- **403 Forbidden:** You're logged in, but you aren't allowed to touch this specific file.
- **404 Not Found:** The classic. The server exists, but the specific page you asked for does not.

5xx: Server Error

This is on them (the website). The server failed to fulfill a valid request.

- **500 Internal Server Error:** A generic "oops" message when the server's code crashes.
- **502 Bad Gateway:** One server got an invalid response from another server it was trying to talk to.
- **503 Service Unavailable:** The server is overloaded or down for maintenance.

API Key

Many REST APIs can only be used after you have registered at the provider.

After registration you receive a so-called API Key.

This key must be included in every http request to the provider.
It can be used to avoid that one user or app sends too many requests or to keep track of the server usage and then calculate the monthly invoice.

- `https://v6.exchangerate-api.com/v6/YOUR_API_KEY/latest/EUR`

  `https://data.fixer.io/api/latest?access_key=YOUR_API_KEY&symbols=USD,GBP,JPY`

API Provider	Free Tier Limit	Authentication Style	Best For...
ExchangeRate-API	1,500 req / month	URL Path	Beginners & reliability
Fixer.io	1,000 req / month	Query Param	High accuracy (ECB data)
Open Exchange Rates	1,000 req / month	App ID Param	Legacy support & simplicity
Alpha Vantage	500 req / day	Query Param	Forex + Stock market data

"Frankfuter API" does not need an API key !

AI is not always right

Code provided by ChatGPT (see https://chatgpt.com/share/69e27dcd-47d0-832d-9dd2-42aa39622727):

ChatGPT

Write a Dart app that finds out the current exchange rate from EUR to USD

Here's a simple Dart console app that fetches the **current EUR → USD exchange rate** using a free API.

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<void> main() async {
  const url = 'https://api.exchangerate.host/latest?base=EUR&symbols=USD';

  try {
    final response = await http.get(Uri.parse(url));

    if (response.statusCode == 200) {
      final data = json.decode(response.body);
      final rate = data['rates']['USD'];

      print('Current EUR → USD rate: $rate');
      print('1 EUR = $rate USD');
    } else {
      print('Failed to fetch exchange rate: ${response.statusCode}');
    }
  } catch (e) {
    print('Error: $e');
  }
}
```

https://api.exchangerate.host/latest?base=EUR&symbols=USD


```
{
  "success": false,
  "error": {
    "code": 101,
    "type": "missing_access_key",
    "info": "You have not supplied an API Access Key.
[Required format: access_key=YOUR_ACCESS_KEY]"
  }
}
```

Using V2 of Frankfurter API

Limit the response to specific target currencies.

```
/* curl -s https://api.frankfurter.dev/v1/latest?symbols=CHF,GBP */
{
  "base": "EUR",
  "date": "2026-03-06",
  "rates": {
    "CHF": 0.9045,
    "GBP": 0.86693
  }
}
```

https://api.frankfurter.dev/v1/latest?symbols=CHF,GBP

Filter target currencies with `quotes`.

```
/* https://api.frankfurter.dev/v2/rates?quotes=USD,GBP */
[
  {
    "date": "2026-04-17",
    "base": "EUR",
    "quote": "GBP",
    "rate": 0.87019
  },
  {
    "date": "2026-04-17",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.1776
  }
]
```

https://api.frankfurter.dev/v2/rates?quotes=USD,GBP

V2 JSON Decoding in detail

https://api.frankfurter.dev/v2/rates?quotes=USD


```
[
  {
    "date": "2026-04-17",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.1776
  }
]
```


```dart
final url = Uri.parse('https://api.frankfurter.dev/v2/rates?quotes=USD');

try {
  print('Fetching current exchange rate...');
  final response = await http.get(url);

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);

    if (data is List<dynamic>) {
      var map = data[0];
      double usdRate = map['rate'];
      String date = map['date'];

      print('------------------------------');
      print('Base Currency: EUR');
      print('Target Currency: USD');
      print('Current Rate: \$${usdRate.toStringAsFixed(4)}');
      print('Last Updated: $date');
      print('------------------------------');
    }
  } else {
    print('Failed to load data. Status code: ${response.statusCode}');
  }
} catch (e) {
  print('An error occurred: $e');
}
```

Historic values

https://api.frankfurter.dev/v2/rates?from=2026-01-01&quotes=USD


```
[
  {
    "date": "2026-01-01",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.1751
  },
  {
    "date": "2026-01-02",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.1738
  },
  {
    "date": "2026-01-03",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.1734
  },
  {
    "date": "2026-01-04",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.1732
  },
  {
    "date": "2026-01-05",
    "base": "EUR",
    "quote": "USD",
    "rate": 1.17
  },
```